

PASO 2 – Estructura de carpetas del bot

Objetivo de esta estructura

- Separar **Discord, lógica, datos y config**
- Que los comandos crezcan sin volverse un quilombo
- Que n8n se integre fácil (HTTP)
- Que sea “profesional” pero sin sobreingeniería

Estructura propuesta

```
vtuber-discord-bot/  
|  
|— bot/  
|   |— __init__.py  
|   |— main.py      # entry point del bot  
|   |— config.py    # tokens, IDs, flags  
|   |  
|   |— commands/  
|       |— __init__.py  
|       |— vtuber.py    # /vtuber buscar, links, imagen, etc  
|       |— agency.py    # /agencia info, vtubers  
|   |  
|   |— services/  
|       |— __init__.py  
|       |— vtuber_service.py # lógica de negocio VTuber  
|       |— agency_service.py # lógica de agencias  
|       |— n8n_client.py    # cliente HTTP hacia n8n  
|   |  
|   |— database/  
|       |— __init__.py  
|       |— db.py        # conexión SQLite  
|       |— models.py    # modelos/tablas
```

```

|   |   | — utils/
|   |   |   | — __init__.py
|   |   |   | — embeds.py      # helpers para embeds
|   |   |   | — logger.py
|   |
|   | — docs/
|   |   | — comandos.md        # lo que ya definimos
|   |   | — arquitectura.md    # diagrama + explicación
|   |
| — .env                      # tokens (NO subir a git)
| — .gitignore
| — requirements.txt
| — README.md

```

Qué va en cada parte (explicado simple)

main.py

- Inicializa el bot
- Registra comandos
- Conecta a Discord
- No tiene lógica de negocio

👉 **Regla:** acá no se hacen búsquedas ni llamadas a APIs.

commands/

Acá vive **la capa Discord**.

Ejemplo:

- recibe `/vtuber buscar ironmouse`
- valida input
- llama a `vtuber_service`
- arma el embed y responde

👉 **No sabe de APIs, DB ni n8n directamente**

services/

Acá está **el cerebro real del bot**.

Ejemplos:

- buscar VTuber
- decidir si usar cache o n8n
- normalizar datos
- combinar info de varias fuentes

Esto te permite:

- cambiar la fuente de datos sin tocar comandos
- testear lógica sin Discord

◆ **n8n_client.py**

Archivo clave.

Responsabilidad:

- hacer requests HTTP a n8n
- recibir JSON
- manejar errores / timeouts

Ejemplo conceptual:

```
get_vtuber_profile(nombre)
get_last_video(nombre)
```

◆ **database/**

Persistencia simple.

- SQLite al principio
 - Cachear:
 - VTubers ya buscadas
 - IDs de YouTube / Twitch
 - Agencia
 - Evita llamadas innecesarias a APIs
-



`utils/embeds.py`

Helpers tipo:

- `vtuber_embed(data)`
- `agency_embed(data)`

👉 Esto evita repetir código y mantiene estilo visual consistente.



Convenciones importantes (guardá esto)

- **Commands NO hablan con DB ni APIs**
- **Services NO saben de Discord**
- **n8n es externo, nunca lógica core**
- **Embeds centralizados**

Si respetás esto, el bot no se rompe cuando crece.